

# IMMO BURUNDI

## COMPLETE BACKEND SYSTEM SPECIFICATION

Part 3 — Sections 82 to 127

|                  |                                                |
|------------------|------------------------------------------------|
| <b>Project</b>   | IMMO BURUNDI                                   |
| <b>Document</b>  | Complete Backend System Specification — Part 3 |
| <b>Sections</b>  | 82 to 127                                      |
| <b>Platforms</b> | Website + Android + iOS + Admin Dashboard      |

# Table of Contents

|                                               |    |
|-----------------------------------------------|----|
| 82. AGENT ANALYTICS . . . . .                 | 4  |
| 83. GLOBAL ANALYTICS . . . . .                | 4  |
| 84. AUDIT LOG . . . . .                       | 4  |
| 85. AUDIT IMMUTABILITY . . . . .              | 4  |
| 86. DATA RETENTION . . . . .                  | 5  |
| 87. BACKUPS . . . . .                         | 5  |
| 88. DISASTER RECOVERY . . . . .               | 5  |
| 89. ADMIN SETTINGS . . . . .                  | 6  |
| 90. COMMISSION CONFIGURATION . . . . .        | 6  |
| 91. CONTRACT SYSTEM . . . . .                 | 6  |
| 92. CONTRACT VERSIONING . . . . .             | 6  |
| 93. LEGAL DOCUMENTS . . . . .                 | 7  |
| 94. CONSENT . . . . .                         | 7  |
| 95. ADMIN DOCUMENT ACCESS . . . . .           | 7  |
| 96. DATA ACCESS PRINCIPLE . . . . .           | 8  |
| 97. PUBLIC API DATA FILTERING . . . . .       | 8  |
| 98. SEARCH RESULT SECURITY . . . . .          | 8  |
| 99. DATABASE CONSTRAINTS . . . . .            | 8  |
| 100. SOFT DELETE . . . . .                    | 9  |
| 101. DATABASE TRANSACTIONS . . . . .          | 9  |
| 102. CONCURRENCY CONTROL . . . . .            | 10 |
| 103. API DOCUMENTATION . . . . .              | 10 |
| 104. API TESTING . . . . .                    | 10 |
| 105. SECURITY TESTING . . . . .               | 11 |
| 106. MONITORING . . . . .                     | 12 |
| 107. ERROR TRACKING . . . . .                 | 12 |
| 108. ENVIRONMENTS . . . . .                   | 12 |
| 109. ENVIRONMENT VARIABLES . . . . .          | 13 |
| 110. DEPLOYMENT . . . . .                     | 13 |
| 111. ADMIN BACKEND MENU . . . . .             | 13 |
| 112. ROLE-SPECIFIC ADMIN DASHBOARDS . . . . . | 14 |
| 113. MAIN ADMIN GLOBAL DASHBOARD . . . . .    | 14 |
| 114. DATA OWNERSHIP . . . . .                 | 15 |
| 115. ACCOUNT DELETION . . . . .               | 15 |

|                                                       |    |
|-------------------------------------------------------|----|
| 116. FRAUD / SUSPICIOUS ACTIVITY . . . . .            | 15 |
| 117. USER REPORTING . . . . .                         | 16 |
| 118. ADMIN ACTION QUEUE . . . . .                     | 16 |
| 119. SCHEDULED JOBS . . . . .                         | 16 |
| 120. PERFORMANCE TARGETS . . . . .                    | 17 |
| 121. SCALABILITY . . . . .                            | 17 |
| 122. FUTURE FEATURES . . . . .                        | 17 |
| 123. CRITICAL BUSINESS RULES . . . . .                | 18 |
| 124. MINIMUM DATABASE TABLE LIST . . . . .            | 19 |
| 125. FINAL BACKEND DEVELOPMENT ORDER . . . . .        | 20 |
| 126. DEVELOPER ACCEPTANCE CRITERIA . . . . .          | 22 |
| 127. MOST IMPORTANT ARCHITECTURAL PRINCIPLE . . . . . | 23 |

## 82. AGENT ANALYTICS

**Track:**

|                |            |
|----------------|------------|
| profile_views  | sales      |
| property_views | rentals    |
| enquiries      | conversion |
| visits         | commission |

## 83. GLOBAL ANALYTICS

**Main Admin dashboard:**

|                      |                       |                     |
|----------------------|-----------------------|---------------------|
| Total users          | Sold properties       | Views               |
| Active users         | Rented properties     | Enquiries           |
| Agents               | Verification requests | Visits              |
| Properties           | Verified properties   | Rental applications |
| Published properties | Promotion orders      | Revenue             |

## 84. AUDIT LOG

|            |
|------------|
| audit_logs |
|------------|

**Fields:**

|               |             |            |
|---------------|-------------|------------|
| id            | resource_id | user_agent |
| actor_user_id | old_data    | request_id |
| action        | new_data    | created_at |
| resource_type | ip_hash     |            |

**Examples:**

|                    |                    |
|--------------------|--------------------|
| PROPERTY_PUBLISHED | PERMISSION_GRANTED |
| PROPERTY_VERIFIED  | REFUND_APPROVED    |
| PAYMENT_CONFIRMED  | STAFF_VIEWED       |
| ROLE_CHANGED       | DOCUMENT_VIEWED    |

## 85. AUDIT IMMUTABILITY

**Ordinary users must not:**

- Edit audit logs
- Delete audit logs
- Disable audit logging

Main Admin may have controlled administrative access, but even administrative actions affecting the audit system must themselves be logged where technically possible.

## 86. DATA RETENTION

Retention periods should be configurable.

**Separate:**

- Active records
- Archived records
- Financial records
- Legal/contract records
- Audit records
- User account data
- Analytics

Do not automatically delete records merely because a user closes an account if they must legally/operationally be retained.

Final retention periods should be reviewed against applicable Burundi requirements.

## 87. BACKUPS

**Database:**

- Automated daily backup
- More frequent backups for critical production data where practical
- Encrypted backups
- Off-site backup
- Backup monitoring
- Restore testing

Do not assume that "backup created" means "backup works."

Perform periodic restoration tests.

## 88. DISASTER RECOVERY

**Developer must document:**

- Recovery Point Objective
- Recovery Time Objective
- Database restore procedure
- File restore procedure
- Server replacement procedure
- Secret recovery procedure
- DNS recovery
- Emergency admin access

## 89. ADMIN SETTINGS

**Main Admin can configure:**

|                     |                        |                   |
|---------------------|------------------------|-------------------|
| Company name        | Commission rates       | SMS provider      |
| Logo                | Rental rules           | Payment providers |
| Contact information | Visit rules            | Map provider      |
| Languages           | Maximum occupants      | Storage provider  |
| Currencies          | Minimum rental advance | Security settings |
| Verification fees   | Notification settings  |                   |
| Promotion prices    | System email           |                   |

Changes must be audited.

## 90. COMMISSION CONFIGURATION

**Commission settings should support:**

|                     |                    |
|---------------------|--------------------|
| sale commission %   | company share %    |
| rental commission % | minimum commission |
| agent share %       | maximum commission |

Rates should be versioned.

Changing a commission rate must not retroactively change existing transactions.

## 91. CONTRACT SYSTEM

Backend should support digital contracts.

contracts

|                 |           |            |
|-----------------|-----------|------------|
| id              | client_id | file_id    |
| contract_number | agent_id  | signed_at  |
| contract_type   | version   | created_at |
| property_id     | status    | updated_at |

**Types:**

|              |           |
|--------------|-----------|
| BROKERAGE    | PROMOTION |
| SALE         | AGENT     |
| RENTAL       | SERVICE   |
| VERIFICATION |           |

## 92. CONTRACT VERSIONING

Never overwrite a signed contract.

**Instead:**

Version 1

Version 2

Version 3

Signed versions become immutable.

**93. LEGAL DOCUMENTS**

**Backend should store configurable versions of:**

- Privacy Policy
- Terms & Conditions
- Verification Disclaimer
- Cookie Policy
- Service Agreement
- Brokerage Agreement
- Rental terms

Record the version accepted by each user.

**94. CONSENT**

user\_consent

|                  |             |
|------------------|-------------|
| id               | accepted    |
| user_id          | accepted_at |
| document_type    | ip_hash     |
| document_version |             |

Important consent actions must be recorded.

**95. ADMIN DOCUMENT ACCESS**

**When a sensitive document is opened:**

**DOCUMENT\_VIEWED**

must be logged.

**This is particularly important for:**

- Identity documents
- Land titles
- Sale agreements
- Financial documents
- Staff private information

## 96. DATA ACCESS PRINCIPLE

**Every API should answer:**

"Does this user have the right to see this specific record?"

**Not simply:**

"Does this user have an admin role?"

**Example:**

**A Provincial Admin may have:**

`properties.view`

**but only for:**

`province_id = assigned province`

## 97. PUBLIC API DATA FILTERING

Never return the complete database object to the frontend.

**Create separate DTOs:**

### PublicPropertyDTO

Contains only public information.

### OwnerPropertyDTO

Contains owner-level information.

### AgentPropertyDTO

Contains agent-level information.

### AdminPropertyDTO

Contains authorized internal information.

This prevents accidental data exposure.

## 98. SEARCH RESULT SECURITY

Search must also apply permissions.

A user should not be able to retrieve private properties simply by knowing the property ID.

## 99. DATABASE CONSTRAINTS

**Use database constraints for:**

- Unique property IDs
- Unique payment references
- Unique verification codes
- Unique promotion codes

- Unique commission codes
- Unique booking references
- Unique agent IDs
- Foreign keys
- Required relationships
- Valid status values

Do not rely only on application validation.

## 100. SOFT DELETE

For important business records use soft deletion.

### Example:

```
deleted_at
deleted_by
deletion_reason
```

### Do not physically delete:

- Financial transactions
- Payments
- Commissions
- Verification history
- Audit logs
- Signed contracts

unless there is a controlled legal/data-retention process.

## 101. DATABASE TRANSACTIONS

Use database transactions for multi-step operations.

### Example:

#### Payment confirmation:



If one critical operation fails, roll back.

## 102. CONCURRENCY CONTROL

### Important for:

- Visit capacity
- Payment processing
- Property publication
- Inventory/capacity
- Commission processing

### Example:

If 10 spots remain and 20 people click Book simultaneously, the backend must never create 20 confirmed bookings.

Use database transactions/locking.

## 103. API DOCUMENTATION

### Developer must generate:

### OpenAPI / Swagger documentation

#### including:

- Endpoint
- Method
- Parameters
- Request body
- Response
- Authentication
- Required permission
- Error codes

## 104. API TESTING

### Automated tests must cover:

#### Authentication

- Login
- Logout
- Password reset
- OTP
- Token refresh

#### Authorization

- Correct role
- Wrong role
- Wrong province
- Wrong commune
- Wrong ownership
- Expired permission

## Properties

- Create
- Submit
- Review
- Publish
- Reject
- Archive

## Verification

- Request
- Payment
- Assignment
- Verification
- Result

## Payments

- Success
- Failure
- Duplicate webhook
- Wrong amount
- Wrong currency

## Visits

- Capacity
- Duplicate booking
- Expired session
- Cancellation

## Rental applications

- Submit
- Review
- Accept
- Reject

# 105. SECURITY TESTING

### Before production:

- Authentication testing
- Authorization testing
- API penetration testing
- File upload testing
- SQL injection testing
- XSS testing
- Rate-limit testing
- Session testing
- Permission escalation testing
- IDOR testing

- Payment manipulation testing

**Especially test:**

Can a normal user change "/properties/123" to "/properties/124" and see another user's private information?

The answer must be NO.

## 106. MONITORING

**Production backend should monitor:**

- API errors
- Database errors
- Server CPU
- RAM
- Storage
- API latency
- Failed login attempts
- Payment failures
- Queue failures
- File upload failures
- Notification failures
- Database connection failures

## 107. ERROR TRACKING

Use a centralized error-monitoring service where appropriate.

**Each error should have:**

|                          |                        |
|--------------------------|------------------------|
| request_id               | environment            |
| user_id if authenticated | error type             |
| endpoint                 | stack trace internally |
| timestamp                |                        |

Never expose internal stack traces to users.

## 108. ENVIRONMENTS

**Maintain separate:**

Development  
Staging  
Production

Never use the production database for development testing.

## 109. ENVIRONMENT VARIABLES

Secrets must not be hard-coded.

**Examples:**

|                      |                    |               |
|----------------------|--------------------|---------------|
| DATABASE_URL         | STORAGE_ACCESS_KEY | EMAIL_API_KEY |
| JWT_SECRET           | STORAGE_SECRET_KEY | MAP_API_KEY   |
| REFRESH_TOKEN_SECRET | PAYMENT_API_KEY    |               |
| REDIS_URL            | SMS_API_KEY        |               |

Use secure secret management.

## 110. DEPLOYMENT

**Recommended:**



Database migrations must be version-controlled.

## 111. ADMIN BACKEND MENU

**Main Admin should see:**

|                     |                     |                 |
|---------------------|---------------------|-----------------|
| Dashboard           | Clients             | Analytics       |
| Users               | Rental Applications | Messages        |
| Staff               | Visit Bookings      | Notifications   |
| Roles & Permissions | Promotions          | Documents       |
| Departments         | Payments            | Contracts       |
| Provinces           | Accounting          | Audit Logs      |
| Communes            | Sales Commissions   | System Settings |
| Properties          | Rental Commissions  | Security        |
| Verification        | Refunds             | Integrations    |
| Agents              | Reports             |                 |

## 112. ROLE-SPECIFIC ADMIN DASHBOARDS

Do not show every module to every employee.

**Example:**

**Accountant**

|                      |                    |
|----------------------|--------------------|
| Dashboard            | Sales Commissions  |
| Payments             | Rental Commissions |
| Verification Revenue | Refunds            |
| Promotion Revenue    | Financial Reports  |

**Verification Officer**

|                    |                      |
|--------------------|----------------------|
| Dashboard          | Documents            |
| Verification Queue | Verification Reports |
| Properties         | Assigned Tasks       |

**Agent**

|                       |             |
|-----------------------|-------------|
| My Properties         | Visits      |
| New Property          | Messages    |
| Verification Requests | Performance |
| Clients               | Commissions |

## 113. MAIN ADMIN GLOBAL DASHBOARD

Main Admin dashboard should contain:

**Operations**

|                  |                      |
|------------------|----------------------|
| Total Properties | Sold                 |
| Published        | Rented               |
| Under Review     | Verification Pending |

**Users**

|        |         |
|--------|---------|
| Users  | Buyers  |
| Agents | Tenants |
| Owners | Staff   |

**Activity**

|           |              |
|-----------|--------------|
| Views     | Visits       |
| Enquiries | Applications |
| Favorites |              |

## Finance

Verification Revenue  
Promotion Revenue  
Sales Commission

Rental Commission  
Refunds  
Net Revenue

## 114. DATA OWNERSHIP

Users can manage their own profile information.

Owners can manage their own property submissions.

Agents can manage their own submissions.

Staff can manage limited profile fields.

Main Admin retains platform-level administrative control.

## 115. ACCOUNT DELETION

**When a user requests account deletion:**



Do not destroy financial/audit records merely because the account was deleted.

## 116. FRAUD / SUSPICIOUS ACTIVITY

**Create:**

suspicious\_activity\_reports

**Possible triggers:**

- Same property repeatedly submitted
- Same documents used across multiple properties
- Unusual payment patterns
- Multiple accounts using same phone
- Abnormal login patterns
- Repeated rejected verification
- Suspicious duplicate listings

These records must be restricted to authorized personnel.

## 117. USER REPORTING

**Users can report:**

- Fake property
- Fraud
- Incorrect information
- Offensive content
- Suspicious agent
- Wrong price
- Duplicate listing

reports

|               |             |             |
|---------------|-------------|-------------|
| id            | reason      | resolution  |
| reporter_id   | description | created_at  |
| resource_type | status      | resolved_at |
| resource_id   | assigned_to |             |

## 118. ADMIN ACTION QUEUE

**Backend should provide task queues for:**

- Properties awaiting review
- Verification requests
- Payment confirmation
- Refund requests
- Reported properties
- Reported users
- Rental applications
- Visit management

## 119. SCHEDULED JOBS

**Automated jobs:**

**Every few minutes**

- Process notifications
- Process payment webhooks
- Process sync queue

**Daily**

- Analytics aggregation
- Expired promotions
- Expired visits
- Reminder notifications
- Backup verification

### Weekly

- Performance summaries
- Financial reports

### Monthly

- Accounting summaries
- Platform statistics

## 120. PERFORMANCE TARGETS

**The backend should be designed for:**

- Fast property search
- Paginated results
- Cached public data
- CDN image delivery
- Database indexing
- Background processing for heavy tasks
- Horizontal scaling later

Do not make the initial system dependent on expensive synchronous operations.

## 121. SCALABILITY

**Architecture should allow future expansion to:**

- More provinces
- More countries
- More currencies
- More languages
- More property types
- More agents
- More payment providers
- More users
- More traffic

Do not hard-code Burundi-specific assumptions into every database table.

## 122. FUTURE FEATURES

**Backend should be designed so the following can be added later:**

- Buyer protection
- Online purchase contracts
- Digital signatures
- Escrow integration
- Mortgage/financing integration
- Property valuation
- Advanced AI price estimation
- Fraud detection

- Map-based search
- 360° property tours
- Video tours
- Agent subscription plans
- Agency accounts
- Developer/property-company accounts
- Investment analytics
- International expansion

## 123. CRITICAL BUSINESS RULES

**The following are NON-NEGOTIABLE:**

### Rule 1

The frontend is never the security layer.

### Rule 2

Backend authorization is mandatory.

### Rule 3

Main Admin has unrestricted access.

### Rule 4

No employee can automatically access all staff private information.

### Rule 5

Agents cannot approve their own properties.

### Rule 6

Verification Officers cannot modify financial records.

### Rule 7

Accountants cannot modify verification results.

### Rule 8

Owners cannot mark their own property Verified.

### Rule 9

Property viewers remain anonymous unless they intentionally interact.

### Rule 10

Financial records must be separated by category.

### Rule 11

Original currency and transaction amount must never be overwritten.

### Rule 12

Verification history must be preserved.

**Rule 13**

Financial history must be preserved.

**Rule 14**

Audit history must be preserved.

**Rule 15**

Critical actions must be traceable to a specific authenticated user.

**Rule 16**

Important operations must use database transactions.

**Rule 17**

Payment webhooks must be verified and idempotent.

**Rule 18**

Visit capacity must be protected against simultaneous bookings.

**Rule 19**

Private documents must never be publicly accessible through predictable URLs.

**Rule 20**

Every permission change must itself be audited.

**124. MINIMUM DATABASE TABLE LIST**

The first production version should contain, at minimum:

|                       |                  |
|-----------------------|------------------|
| users                 | role_permissions |
| roles                 | user_roles       |
| permissions           | user_permissions |
| staff_profiles        | communes         |
| departments           | zones            |
| provinces             |                  |
| agents                |                  |
| clients               |                  |
| properties            |                  |
| property_media        |                  |
| property_documents    |                  |
| verification_requests |                  |
| verification_checks   |                  |

favorites  
property\_views  
property\_daily\_analytics  
property\_shares

enquiries  
conversations  
conversation\_participants  
messages

visit\_sessions  
visit\_bookings

## rental\_applications

promotion\_packages  
promotion\_orders

|                   |                       |
|-------------------|-----------------------|
| payments          | rental_commissions    |
| transactions      | refunds               |
| sales_commissions | financial_adjustments |

contracts  
user\_consent

## files

notifications  
notification\_preferences

## reports

### audit\_logs

suspicious\_activity\_reports  
user\_reports

system\_settings  
exchange\_rates

## 125. FINAL BACKEND DEVELOPMENT ORDER

The developer should implement in approximately this order:

### Phase 1 — Foundation

1. PostgreSQL
2. Database migrations
3. Users
4. Authentication

- 5. Roles
- 6. Permissions
- 7. Authorization middleware
- 8. Audit logging

## **Phase 2 — Core property system**

- 9. Locations
- 10. Properties
- 11. Property media
- 12. Property documents
- 13. Agents
- 14. Property workflow
- 15. Search

## **Phase 3 — Verification**

- 16. Verification requests
- 17. Verification codes
- 18. Verification checks
- 19. Verification dashboard
- 20. Verification history

## **Phase 4 — Client services**

- 21. Favorites
- 22. Enquiries
- 23. Messaging
- 24. Visit bookings
- 25. Rental applications

## **Phase 5 — Money**

- 26. Payment infrastructure
- 27. Verification payments
- 28. Promotion payments
- 29. Accounting
- 30. Sales commissions
- 31. Rental commissions
- 32. Refunds
- 33. Financial reports

## **Phase 6 — Analytics**

- 34. Property views
- 35. Analytics
- 36. Agent performance
- 37. Global dashboard
- 38. Reports

## **Phase 7 — Operations**

- 39. Notifications
- 40. Staff management

- 41. Admin dashboards
- 42. Contracts
- 43. Document management

## Phase 8 — Advanced

- 44. Offline synchronization
- 45. Duplicate detection
- 46. Smart price estimator
- 47. Fraud detection
- 48. Advanced analytics

## Phase 9 — Security & production

- 49. Security testing
- 50. Load testing
- 51. Backup/restore testing
- 52. Monitoring
- 53. CI/CD
- 54. Production deployment

# 126. DEVELOPER ACCEPTANCE CRITERIA

**The backend is not considered complete until:**

- Authentication works
- RBAC works
- Permissions are enforced server-side
- Geographic scopes work
- Property IDs are generated automatically
- Property workflow works
- Verification workflow works
- Verification codes work
- Private documents are protected
- Payment references are unique
- Accounting categories are separated
- Commission systems work
- Visit capacity cannot be exceeded
- Rental applications are private
- Property analytics work
- Viewer identity remains protected
- Notifications work
- Audit logs work
- Reports work
- Offline sync works
- Backups work
- Restore testing succeeds
- API documentation exists
- Automated tests exist

- Security testing is completed
- Production monitoring is configured

## 127. MOST IMPORTANT ARCHITECTURAL PRINCIPLE

IMMO BURUNDI should not be built as a simple website with an admin panel.

It should be built as a secure real-estate platform with a centralized backend and permission system.

**The architecture must treat:**

Properties + Verification + Users + Money + Documents + Permissions + Audit Logs  
as interconnected but separately protected systems.

The frontend should consume the backend through documented APIs.

**The backend should remain the ultimate authority over:**

- Who can see something
- Who can change something
- Who can approve something
- Who can publish something
- Who can access money
- Who can access private documents
- Who can access staff information
- Who performed an action
- When the action happened